\GITHUB AGENTIC WORKFLOWS

Conceptos Preliminares



De Codespaces a un Issue creado por un agente de IA, paso a paso con gh-aw.

github.github.com/gh-aw

MAPA DEL LABORATORIO

Trece conceptos, cuatro etapas

1

Entorno

- GitHub Codespaces
- GitHub CLI (gh)
- GitHub Agentic Workflows (gh-aw)

2

Credenciales

- Fine-grained Personal Access Token
- GitHub Actions Secrets

3

Archivos del agente

- Carpeta .github
- Markdown (.md) y Frontmatter
- Compilación y .lock.yml

4

Ejecución

- Triggers (workflow_dispatch)
- GitHub Actions
- Safe Outputs
- GitHub Issues

GitHub Codespaces

Qué es un Codespace

Un entorno de desarrollo completo en la nube, listo para usar.

Cómo se crea

En el repositorio: Code → Codespaces → Create codespace.

VS Code en el navegador

El mismo editor, sin instalar nada en tu máquina.

Terminal integrada

Acá ejecutamos gh y gh aw durante todo el laboratorio.

Relación con el repositorio

El Codespace se crea a partir del repo y trabaja sobre su contenido.



GitHub CLI (gh)

La CLI oficial para interactuar con GitHub desde la terminal. En el laboratorio usamos tres comandos:

1

```
gh --version
```

Verifica la instalación de GitHub CLI.

2

```
gh auth status
```

Muestra el estado de autenticación.

3

```
gh extension install
github/gh-aw
```

Instala gh-aw con el administrador de extensiones de gh.

Idea clave: gh-aw no es una herramienta aparte: es una extensión de GitHub CLI.

CONCEPTO 3 · ENTORNO

GitHub Agentic Workflows (gh-aw)

Qué es

- Extensión de GitHub CLI que define y ejecuta workflows basados en agentes dentro de GitHub Actions.
- Se escriben en Markdown.
- Pueden usar motores de IA como GitHub Copilot.

COMANDOS USADOS EN EL LABORATORIO

```
gh aw version
```

Comprueba la versión instalada.

```
gh aw compile .github/workflows/hello-agent.md
```

Compila la definición Markdown y genera el .lock.yml ejecutable por GitHub Actions.

También existen (no los usamos hoy)

`gh aw init`	`gh aw new`	`gh aw add`	`gh aw add-wizard`	`gh aw list`
`gh aw run`	`gh aw status`	`gh aw logs`	`gh aw doctor`	

Documentación:
github.github.com/gh-aw

CONCEPTO 4 · CREDENCIALES

Fine-grained Personal Access Token

PAT

Personal Access Token: una credencial para autenticarse en GitHub.

Fine-grained

Permite restringir con mucha más precisión qué permisos tiene el token.

`COPILOT_GITHUB_TOKEN`

El secreto que usamos para que el Agentic Workflow se autentique contra GitHub Copilot.

Por qué importa: introduce la autenticación del agente, no solamente el uso de un PAT en general.

GitHub Actions Secrets

A nivel de repositorio, creamos el secreto en:

```
Settings → Secrets and variables → Actions → COPILOT_GITHUB_TOKEN
```

El PAT

Es la credencial.

El Secret

Es el lugar donde guardamos esa credencial de forma segura.

GitHub Actions

Es quien la utiliza al ejecutar el workflow.

Para remarcar en clase: el PAT es la credencial; el Secret es dónde se almacena.

La carpeta .github

```
.github/
└── workflows/
    ├── hello-agent.md
    └── hello-agent.lock.yml
```

`.github`

Carpeta especial que GitHub usa para distintos archivos de configuración del repositorio.

`workflows/`

Es donde GitHub espera encontrar las definiciones de workflows.

Nuestros dos archivos

hello-agent.md (la fuente) y hello-agent.lock.yml (el workflow generado).

Agentic Workflow Markdown

`hello-agent.md`

PARTE 1

Frontmatter

Configura el workflow

+

PARTE 2

Instrucciones

En lenguaje natural, para el agente

Es el archivo fuente

Es la definición del Agentic Workflow: el Markdown es lo que se mantiene y se edita.

El frontmatter configura

Nombre, trigger, permisos y salidas del workflow.

El cuerpo le habla al agente

Las instrucciones que el agente va a seguir, escritas en lenguaje natural.

Frontmatter

```
---
name: hello-agent

on:
  workflow_dispatch:

safe-outputs:
  create-issue:
---
```

Después del frontmatter vienen las instrucciones en lenguaje natural.

Nombre

name: cómo se llama el workflow.

Trigger

on: cuándo se inicia la ejecución.

Permisos y capacidades

Qué puede hacer el agente.

Safe Outputs

safe-outputs: las salidas controladas que declaramos.

Triggers: workflow_dispatch

```
on:
  workflow_dispatch:
```



Significa:

El workflow puede iniciarse manualmente.

Un trigger define cuándo se ejecuta el workflow. Es una buena primera introducción, sin sumar otros eventos todavía.

QUEDAN PARA MÁS ADELANTE

```
schedule:
```

```
issues:
```

```
pull_request:
```

Hoy usamos solamente uno.

Compilación: del .md al .lock.yml



`.md = fuente`

La fuente del Agentic Workflow: lo que escribimos y editamos.

`.lock.yml = generado`

El workflow completo de GitHub Actions que se produce al compilar.

Dato importante: las instrucciones Markdown se cargan en runtime; la compilación construye la configuración ejecutable del workflow.

CONCEPTO 11 · EJECUCIÓN

GitHub Actions

Es donde el alumno descubre cómo se ejecuta el agente. Recorrido en el laboratorio:

1

Pestaña Actions

Vemos los workflows del repositorio.

2

Workflow

Elegimos hello-agent.

3

Run workflow

Lo iniciá el trigger workflow_dispatch.

4

Ejecución

El agente corre en GitHub Actions.

5

Logs y estado

Revisamos qué pasó y cómo terminó.

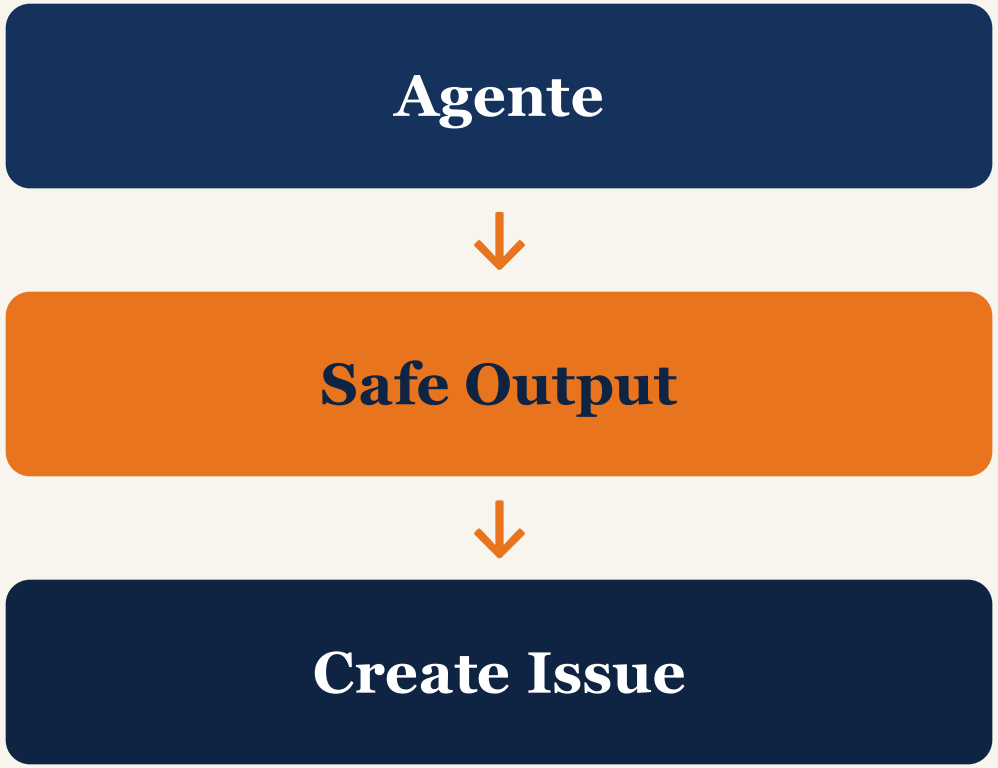
Conceptualmente: GitHub Actions es la infraestructura donde se ejecuta el Agentic Workflow.

Safe Outputs

```
safe-outputs:
  create-issue:
```

No es "el agente puede hacer cualquier cosa".

Declaramos una salida controlada: el agente solo puede producir lo que el workflow permite.



GitHub Agentic Workflows enfatiza guardrails, permisos y revisión de los resultados que producen los agentes.

CONCEPTO 13 · EJECUCIÓN

GitHub Issues

Usamos un Issue como resultado observable del agente: el alumno ve de inmediato qué produjo.



Didácticamente: todo el recorrido del agente termina en algo que se puede abrir y leer.

CIERRE

Cuatro conceptos que convierten el laboratorio en uno de Agentic Workflows



Frontmatter

Configura el workflow: nombre, trigger, permisos y salidas.



Trigger

workflow_dispatch: cuándo y cómo se inicia la ejecución.



Compilación

.md → .lock.yml con gh aw compile.



Safe Outputs

Salidas controladas: el agente actúa dentro de guardrails.

Documentación: github.github.com/gh-aw